

The cert-machine method: three-valued decisions with witnesses in exact and interval arithmetic, and the verification discipline that keeps them sound

Carlos Toledo*

October 2026 — draft v0.1.1 for review (2026-10-09; v0.1 the same morning, before the
headline repairs of Section 8); every number interpolated from the record;
machine-derived, not peer-reviewed; author list to be agreed

Abstract

cert-machine is a system for deciding the numbers and claims that machines now propose — constants, counterexamples, packings, fitted distributions, operating points, environmental contours, formal certificates. Its one description is *Independent exact certification of machine-generated mathematics — exact arithmetic, no code shared with the claimant, refusal as a verdict*. This paper sets out the mathematics and the verification behind it. A claim is read as a predicate over a declared box of inputs and receives one of three verdicts: CERTIFIED when finitely many exact or outward-rounded computations show the predicate at every point of the box, REFUTED when they show its negation, with the falsifying object, and REFUSED otherwise — where the declared evidence itself is split, with one proved witness on each side and the value of a declared quantity at which the verdict would change. The verdicts are proved sound under the inclusion property of interval extensions; the arithmetic that secures that property on IEEE-754 doubles and on exact rationals is described; and so are the certifiers that reduce analytic questions to finitely many such decisions: the Krawczyk operator for the existence and uniqueness of a zero, a second-order test for a maximum over a box, a monotonicity proof for a supremum at a family’s limit, monotone and multilinear reductions of universal statements over boxes, floating-point filters with exact integer fallbacks for geometric signs, exact rational sums of squares, and certified minima of cosine polynomials. The discipline that keeps the system sound is enforced by code that refuses: screens may only prune, red controls must fire, bytes are pinned, decisions are re-derived by second implementations that share no code, and published records re-run in a reader’s browser. The evidence is a register of 132 published machine-generated claims (107 certified, 2 refuted, 9 partial, 5 mixed, 6 repaired, 2 needing data, 1 queued), an engineering decision procedure and a certified extreme-value atlas of 46,602 fits; one of the laboratory’s own theorems has been re-certified by an outside group with no shared code. The contribution claimed is the system — the grammar, the certifiers that serve it, the discipline that guards it and the public record — and not interval arithmetic, which is standard.

*Independent researcher. Contact: carlos@carlostoledo.co. This work was produced with substantial AI assistance in a certified-verification setting: constructions and exposition were proposed with the help of large language models, and every quantitative claim is decided by outward-rounded interval arithmetic or exact rational arithmetic, with adversarial controls that must fire for the certificate to be accepted. The certificates, not the models, are the evidence, and they are public and re-runnable. Machine-derived and not peer-reviewed.

1 Introduction

Machines now propose mathematics and engineering numbers at a rate no referee can read. Conjectures about fundamental constants are generated from numerical coincidences and supported by a probability argument [29]; constructions in combinatorics and analysis are found by program search [31, 18]; long proofs arrive with machine formalisations [22]; and in engineering, surrogate models, optimisers and language models propose operating points and design values [24]. Each arrives with its producer’s checker. A checker written by the producer shares the producer’s blind spots, and the published record shows the cost: a floating-point product printed as an exact constant, witnesses that hold only within a verifier’s tolerance, a checker that accepts what its own definition excludes, and a clause present in a paper’s theorem and absent from the formal statement that was proved [30].

cert-machine decides such claims independently. It reads the claim from its published statement and bytes, never runs the claimant’s program in the trust path, decides it in exact rational arithmetic or in outward-rounded interval arithmetic, and publishes one of three verdicts with the object that justifies it. Where it cannot decide, it says so, and the refusal is recorded like any other result. This paper is the method paper of the system. Section 2 defines the verdicts and proves them sound; Section 3 describes the arithmetic that secures the hypothesis of that proof; Section 4 the certifiers that reduce the questions machines actually ask — is this the maximum, does this hold for every input in the envelope, is this point inside this polygon, is this polynomial nonnegative — to finitely many such decisions; Section 5 the verification discipline, enforced by code that refuses; Section 6 the record that is the system’s evidence; and Sections 7–8 its relation to existing practice and its limits.

The design choice that runs through all of it is that the system is sound and deliberately incomplete. A procedure that must always answer cannot be both, for the questions machines propose include undecidable ones [2]; a procedure that may refuse can. The cost of soundness is paid in refusals, and the system’s answer to that cost is to make every refusal informative: it names its reason, and, where the declared evidence is split, the witnesses and the threshold that would decide it.

The contribution claimed is the system: (i) a three-valued decision grammar with witnesses, refusal as a result and flip thresholds, with its soundness stated and proved (Section 2); (ii) a set of certifiers, each standard in its field, assembled so that the claims that arrive — from mathematics, from engineering and from metocean statistics — are decided by one grammar (Sections 3–4); (iii) a verification discipline whose rules are lines that make a build refuse rather than sentences that advise (Section 5); and (iv) a public record that any reader can re-run, in which every number of every companion paper is a macro written from a certificate (Section 6). Interval arithmetic, the Krawczyk operator and exact geometric predicates are decades old [16, 10, 33, 34]; nothing here claims them.

2 The decision grammar

2.1 Claims over boxes

A *box* is a product $X = [\underline{x}_1, \bar{x}_1] \times \cdots \times [\underline{x}_n, \bar{x}_n]$ of closed intervals with rational endpoints; a double is a rational, so a box of doubles is a box. A *declared envelope* B is a box each of whose sides carries the source of its bounds — a survey, a data sheet, a chart. A number someone printed is the box its digits allow: a printed 0.0634 is $[0.06335, 0.06345]$, and a fit printed to four

decimals is a box in parameter space.

Definition 1 (interval extension). Let $g : D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$. A map G from boxes in D to intervals is an *interval extension* of g if $g(X) \subseteq G(X)$ for every box $X \subseteq D$ (the inclusion property).

The inclusion property is the one hypothesis of every verdict below, and Section 3 is about securing it. The extensions used are natural extensions — the formula evaluated in interval arithmetic — and the classical theorem that a composition of operations with the inclusion property has it [16, 17] carries it from the primitives to the model.

Definition 2 (claim). A *claim* is a pair (B, φ) of a box and a predicate $\varphi(x) \equiv \bigwedge_j g_j(x) \leq c_j$ whose functions have interval extensions G_j (an equality, or an exact predicate such as the orientation of three points, is the special case decided in exact arithmetic). It is read in one of two ways:

universal the claim asserts $\varphi(x)$ for every $x \in B$ — the reading of a mathematical statement (“for all $n \leq N$ ”, “on $[0, 1]$ ”, “for every member of this family”);

epistemic the claim asserts $\varphi(x^\dagger)$ for an unknown true input $x^\dagger \in B$ — the reading of an engineering number, where B is the declared uncertainty about the input.

The two readings differ in what a single proved counterexample does. Under the universal reading it refutes the claim; under the epistemic reading it shows only that the declared evidence does not settle the question.

2.2 Three verdicts

Definition 3 (verdicts). Let $\mathcal{X}$ be a finite cover of B by boxes. The claim is

certified if $\sup G_j(X) \leq c_j$ for every j and every $X \in \mathcal{X}$;

refuted under the universal reading, if some box $X^* \subseteq B$ (typically a thin box, a point) has $\inf G_j(X^*) > c_j$ for some j ; under the epistemic reading, if every $X \in \mathcal{X}$ has $\inf G_j(X) > c_j$ for some j , or the claim contradicts a bound that holds on all of B ;

refused otherwise. Under the epistemic reading a refusal is *split* when two points $x^-, x^+ \in B$ have been proved to satisfy and to violate φ respectively, and *unresolved* when the enclosures did not separate within the budget.

Proposition 1 (soundness). *If every G_j is an interval extension of g_j , then CERTIFIED implies $\varphi(x)$ for every $x \in B$; REFUTED implies, under the universal reading, $\neg\varphi(x)$ for some $x \in B$ and, under the epistemic reading, $\neg\varphi(x)$ for every $x \in B$; and a split REFUSED implies that neither $\forall x \in B \varphi(x)$ nor $\forall x \in B \neg\varphi(x)$ holds, so that $\varphi(x^\dagger)$ is not determined by the declared envelope.*

Proof. Let $x \in B$ and choose $X \in \mathcal{X}$ containing it. By the inclusion property $g_j(x) \in g_j(X) \subseteq G_j(X)$, hence $g_j(x) \leq \sup G_j(X) \leq c_j$ for every j , which is $\varphi(x)$. The refutations are the same argument with the inequality reversed, on X^* or on every member of the cover. For a split refusal, x^- and x^+ are points of B at which φ is proved true and false respectively; either could be $x^\dagger$. $\square$

An unresolved refusal claims nothing, and the grammar keeps it that way: a refusal is terminal, never retried at lower rigour, never converted into a probability and never dropped from the record [30]. The only path from REFUSED to CERTIFIED is a new exact argument — a finer cover, a better extension, a different certifier. The three words are not probabilities and there is no fourth.

2.3 Witnesses

Every verdict carries the object that justifies it, and the object is what is published. A certification carries its *certificate*: the inputs pinned by digest, the cover, the enclosures, and a statement of what would make it fail. The certificate constructor of the interval toolkit refuses to build a certificate whose falsifier is not stated and a CERTIFIED that carries no evidence, on the principle that a certificate that cannot go red is decoration. A refutation carries the falsifying object, printed — a counterexample, an observation outside a region, a violated equation, a digit that is not the rounding of an enclosure. A split refusal carries its two witnesses. A certificate re-runs without the engine: its facts are exact or outward-rounded, and any arithmetic that rounds outward can check them.

2.4 Thresholds

A split refusal is the most useful outcome the grammar has, because the question it leaves open has a size.

Definition 4 (flip thresholds). Let the envelope depend on a declared scalar p — an operating choice, or one end of an interval that a measurement could narrow — through boxes $B(p)$, and let the verdict be monotone in p : if it is CERTIFIED at p it is CERTIFIED at every $p' \leq p$, and if it is REFUTED at p it is REFUTED at every $p' \geq p$. The *flip thresholds* are $p_C = \sup\{p : \text{CERTIFIED over } B(p)\}$ and $p_R = \inf\{p : \text{REFUTED over } B(p)\}$.

Proposition 2. *Under monotonicity, if the procedure has certified the claim over $B(p)$ and refuted it over $B(\bar{p})$, then the claim is CERTIFIED for every $p \leq \underline{p}$ and REFUTED for every $p \geq \bar{p}$, and $\underline{p} \leq p_C \leq p_R \leq \bar{p}$.*

The proof is the definition. The procedure computes $\underline{p}$ and $\bar{p}$ by bisection over rational p , each endpoint decided like any other claim, or in closed form where p enters additively, the result rounded outward to the precision reported; the engineering demonstrator does both, the pump discharge in closed form because the wellhead pressure is the discharge plus a model term, and two inspection thresholds by bisection [24]. The threshold turns a refusal into a measurement or an operating change with a known effect on the verdict: at this discharge the rule is proved, at that roughness it is refuted.

2.5 Compositions

A receipt with several checks is REFUTED if any check is refuted, otherwise REFUSED if any is refused, otherwise CERTIFIED; by Proposition 1 this is sound for their conjunction. The public register composes the three verdicts into words that each say what they are made of (Table 1); none is a fourth verdict. Two consequences are enforced rather than recommended [30]. A scope sentence is load-bearing, because a CERTIFIED without the scope beside it can say more than the record, as when the computational fragment of a claim is certified and its analytic core is

word	composed of	the witness the row carries
CERTIFIED	—	a certificate with its falsifier; re-runs without the engine
REFUTED	—	the falsifying object, printed
REFUSED	—	the reason; for a split refusal, two witnesses and the thresholds
PARTIAL	certified + refused	which fragment certified and which core was not reached
NEEDS DATA	refused, cause with the claimant	the bytes that would decide the row, named
MIXED	certified + refuted, one record	the counts of both, and the items refuted
REPAIRED	refuted + certified	the object as printed refuted; the object beside it certified
QUEUED	not a verdict	nothing; the row is not counted as decided

Table 1: The verdicts and their compositions in the public register [30]. Engineering surfaces write PROVED for CERTIFIED; a ranking of families says DECIDED; the words for a printed number — REPRODUCED, CONSISTENT, OFF THE MAXIMUM, OUTSIDE ITS SUPPORT — refine the three for a comparison of two boxes [23].

not touched. And the kinds of refusal are never summed: the instrument declining, the claimant having published nothing, a budget exhausted, a cell whose box did not close and an audit stating its reach are different behaviours with different denominators.

3 The arithmetic

Proposition 1 has one hypothesis, the inclusion property, and the arithmetic exists to secure it. Two kinds are used, and the choice is made per quantity: exact rationals where a tie must be decided or the object is rational, outward-rounded intervals where the quantity is transcendental or the data are long.

Exact rationals. Numbers are reduced fractions of arbitrary-precision integers. Doubles convert to them losslessly; linear systems are solved by exact elimination, which reports a singular matrix rather than a small pivot; conversion back to a double is for display only. Rational arithmetic decides every sign it is asked, and it is used for the corners of multilinear forms (Section 4.4), the expected counts of a benchmark, polynomial identities, and the ends of enclosures whose comparison must be exact. Decimal literals — the digits a claimant printed — are read as the rationals they denote and kept as strings beside their double images, so that a vertex printed to three decimals and an observation printed to four can be found to coincide [23].

Outward rounding on doubles. IEEE-754 arithmetic in round-to-nearest returns, for each of $+$, $-$, $\times$, $\div$, the double nearest the exact result, which therefore lies within half a unit in the last place of the computed one [8, 5]. JavaScript cannot set the rounding mode, so directed rounding is emulated: each computed endpoint is moved one unit in the last place outward by stepping its bit pattern. The trust base of this emulation is stated once: the correctly rounded $+$, $-$, $\times$, $\div$, $\sqrt{}$ that IEEE-754 mandates and V8 implements, V8’s arbitrary-precision integers, and nothing in `Math.*`, which ECMA-262 leaves “implementation-approximated” (a 2017 proposal to require one-ulp accuracy was closed without adoption); the Node version that produced every record is

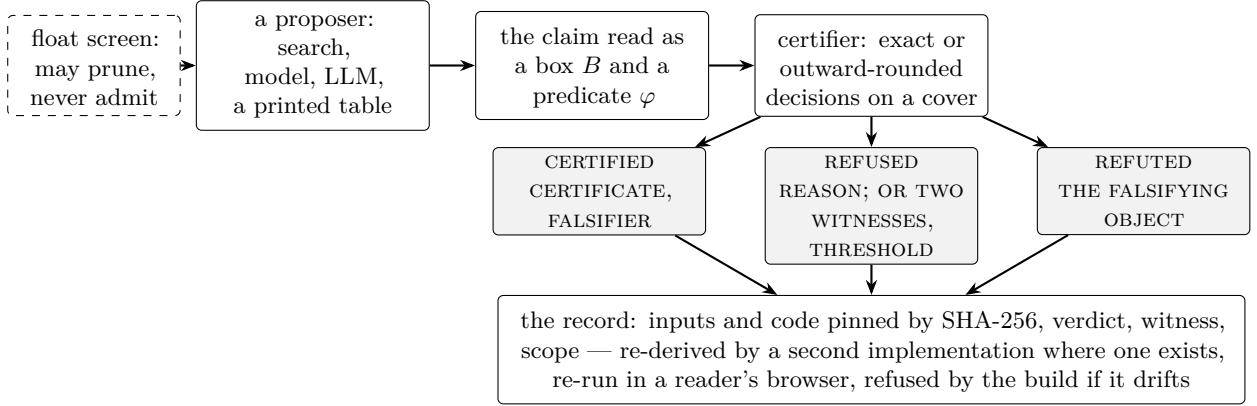


Figure 1: The system. A proposer is never opened and never trusted; its output is read as a claim over a box and decided by a certifier; the verdict, its witness and the pinned inputs form a record that re-runs without the engine.

written into the record (v24.14.1, V8 13.6, whose `Math.*` is the `fdlibm` port, faithful to within one ulp and not correctly rounded; V8 moved these functions to a correctly rounded library in May 2026, which is why the version is material).

Proposition 3. *Let $\circ \in \{+, -, \times, \div\}$, let A, B be intervals of doubles — pairs of non-NaN doubles with lower end at most upper end, an infinite endpoint allowed as a bound — with $0 \notin B$ when $\circ = \div$, and let ℓ and h be the round-to-nearest values of the smallest and largest of the endpoint combinations $a \circ b$. If no endpoint combination is indeterminate ($0 \cdot \infty, \infty - \infty$), then $A \circ B \subseteq [\text{down}(\ell), \text{up}(h)]$, where down and up step one double toward $\mp\infty$ and fix $\mp\infty$; an exact result beyond the largest double rounds to $\pm\infty$, which the step fixes, so overflow is enclosed.*

Proof. For these operations the extremes of $a \circ b$ over a product of intervals are attained at endpoint combinations (monotonicity in each argument on each sign region, with $0 \notin B$ for division). The exact extreme lies within half the spacing of the doubles on its side of its round-to-nearest value, and the adjacent double on that side is a whole spacing away (subnormal spacing included), so it lies on the correct side of $\text{down}(\ell)$ and $\text{up}(h)$. $\square$

The domain of the proposition is enforced, not assumed: a malformed operand (a NaN, an inverted pair) or an indeterminate result is refused by exception and never decided, because a consumer written as “not proved above the bar, hence below it” would otherwise certify on a NaN (until 2026-10-09 nothing checked this, and the batteries skipped non-finite results; enforcing it exposed one battery check that had been passing on malformed boxes). Division by an interval that contains zero is refused, not extended; a square is clamped at zero; a non-integer power is refused, and so is an integer exponent of 2^{31} or more, where the implementation’s exponent arithmetic would wrap. The widening costs about one bit per operation and nothing at the accuracy the claims need; the domain checks cost a few comparisons per operation (ten million operations in 0.18 s). Where more precision is required — a constant to be certified to twelve digits, a Catalan or Li_2 value — a separate arbitrary-precision arithmetic carries a binary mantissa of arbitrary length with true directed rounding (floor and ceiling of the shifted integer), and converts its results to doubles outward with an exact check.

Elementary and special functions. The certifiers of this paper trust no library function, because the platform’s `Math.exp` and `Math.log` carry no accuracy guarantee in ECMAScript — with one exception that Section 8 names: an older helper that encloses the platform’s cosine by a derived pad rather than by a series is still called from six files, among them the radii-polynomial certifier of Section 4, and its retirement for the series below is scheduled with the re-run of those records. Each function is a truncated series evaluated in interval arithmetic plus an explicit bound on the omitted tail: `exp` after reduction $x = k \ln 2 + r$, $|r| < 1$, by twenty-four Taylor terms and the tail $|r|^{25}/25! (1 - |r|/26)^{-1}$; `log` after $x = m 2^e$ by the series of `atanh` in $v = (m - 1)/(m + 1) \in [0, 1/3)$, whose one-sided tail is valid because $v \geq 0$; `sin` and `cos` after reduction by an interval enclosure of $\pi/2$, with the box’s enclosure forced to ± 1 wherever it may contain an extremum; `ln 2` and π in exact rationals by `atanh(1/3)` and Machin’s formula, converted outward. Monotone functions on wide boxes are evaluated at the endpoints. The special functions follow the same pattern: Γ by Spouge’s formula with its error bound taken ten times over; $\ln \Gamma$, ψ and ψ' by Binet’s expansion after shifting the argument above twenty, the first omitted term as the bound; the regularized incomplete gamma function by its series with a geometric tail; Bessel functions by their power series with a certified ratio bound; `erfc` by Laplace’s continued fraction, whose consecutive convergents bracket the value [1]; and inverses (Φ^{-1}, P^{-1}) by bisection that moves an end only on a certified sign. The tails are computed in round-to-nearest and are many orders below the one-ulp widening that absorbs them; that margin, not the tail’s own rounding, is what makes them safe, and it is stated where it is used.

Long sums. A running interval sum rounds outward at each of n additions, so its width grows linearly in n ; at the 175,320 hourly values of one buoy it is a thousand times what the terms carry. Each endpoint stream is instead summed by the compensated `Sum2` algorithm and the error bound of [21] is added outward with a factor of two to spare, which is what makes a likelihood over a hundred thousand data an enclosure narrow enough to rank families by [23].

Derivatives. Gradients and Hessians that a certifier needs come from second-order forward differentiation over the same interval arithmetic (a value, a gradient and a Hessian carried through every operation), or from univariate Taylor jets, so that the derivative a proof uses is an extension of the function the proof is about. The remaining families carry hand-written scores and Hessians, evaluated in the same interval arithmetic.

4 The certifiers

Each certifier below turns a question that is analytic in form into finitely many decisions of the kind Proposition 1 covers. None is new; the assembly is, together with the rule that each refuses rather than guesses.

4.1 A zero, exactly one

Theorem 1 (Krawczyk [10, 17, 33]). *Let $f : D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^n$ be continuously differentiable, $X \subseteq D$ a box, $\hat{x} \in X$, Y a nonsingular matrix and $F'(X)$ an interval extension of the Jacobian over X . If*

$$K(X) = \hat{x} - Yf(\hat{x}) + (I - YF'(X))(X - \hat{x}) \subseteq \text{int } X,$$

then f has exactly one zero in X .

The certifier takes $\hat{x}$ from a floating-point search, Y as an approximate inverse of the Jacobian at $\hat{x}$, evaluates $K(X)$ with every datum enclosed by its neighbouring doubles, and inflates X for a bounded number of rounds; if the inclusion never holds the zero is REFUSED with the reason “no contraction”. A sibling certifier for fixed points in weighted sequence spaces uses the radii-polynomial form of the same argument and requires the contraction factor to be proved below one before it states uniqueness.

4.2 A maximum in a box

Proposition 4. *Let ℓ be twice continuously differentiable on a convex box X , let $\nabla\ell$ have exactly one zero θ^* in X , and let the Hessian of ℓ be negative definite at every point of X . Then θ^* is the unique maximiser of ℓ on X .*

Proof. A function whose Hessian is negative definite on a convex set is strictly concave there, and a critical point of a strictly concave function is its unique maximiser on the set. $\square$

Negative definiteness over the box is proved one of two ways: by Sylvester’s criterion on the interval Hessian, its leading principal minors enclosed and proved of alternating sign over all of X (for up to three parameters, with the two enclosures of each symmetric entry intersected); or, where the box’s minors are too wide, at the candidate point together with a proof that every Hessian over X is nonsingular — $|I - AH(X)|u < u$ componentwise for a positive vector u and an approximate inverse A — so that, X being connected and the eigenvalues continuous, no Hessian over X can change its inertia. This is how a maximum-likelihood fit becomes a certificate [23]. The statement is local: that no better maximum exists elsewhere in the family remains the search’s claim, and every fit is stated with that scope.

4.3 A supremum at a limit

Some families have no interior maximum on some data: the likelihood rises toward a limit member, and an optimiser prints the last iterate as the fit.

Proposition 5. *Let $\ell(\mu, \sigma, Q)$ be continuous on $B \times [0, q_1]$ and differentiable in Q on $B \times (0, q_1]$, and let $\partial\ell/\partial Q < 0$ there. Then for every $(\mu, \sigma) \in B$ and every $Q \in (0, q_1]$, $\ell(\mu, \sigma, Q) < \ell(\mu, \sigma, 0)$.*

The proof is the mean value theorem. With $Q = 1/\sqrt{\alpha}$ the generalized gamma reaches the lognormal at $Q = 0$ [28], and in these coordinates $Q = 0$ is an ordinary point; a proof that the Q -score is negative over a neighbourhood of the lognormal fit refuses the family at its edge *with the proof* that every member there is less likely than its limit. A threshold on α in place of this proof hands a different family the verdict at 544 of 7,767 cell-blocks of the extreme-value atlas [23], which is the cost of a threshold measured.

4.4 A statement for every input in a box

Three reductions decide a universal statement over a box.

Bisection with a proposer. The box is split until each piece’s enclosure decides, a floating-point heuristic choosing the coordinate to split and never a verdict; a depth budget bounds the work, and an exhausted budget is an unresolved refusal. To publish a split refusal, the corners are evaluated as thin boxes and one proved witness is kept on each side.

Monotone implicit functions. Where a model quantity is the root of an equation monotone in its parameters, the root's range over the box is bracketed by the roots at two corners, and each end counts only when the equation's sign just beyond it is proved. Colebrook's friction law is handled this way: with $x = 1/\sqrt{f}$ the root of $h(x) = x + (2/\ln 10) \ln(a + bx)$, increasing in x , a and b , every root over the box lies in $[x^*(a_+, b_+), x^*(a_-, b_-)]$, and a floating-point Newton iteration only locates the ends [24].

Multilinear forms.

Lemma 1. *Let f be multilinear on a box $X \subset \mathbb{R}^n$ (affine in each variable when the others are fixed). Then $f(X) = [\min_v f(v), \max_v f(v)]$, the minimum and maximum taken over the 2^n vertices of X .*

Proof. Fix all variables but one; f is affine in it, so its extremes over that side are at the side's ends. Induct over the variables to reach a vertex for each extreme. f is continuous and X connected, so every value between is attained. $\square$

An emissions-abatement estimate, a sum of products of premises each declared as an interval, is such a form. The kernel checks multilinearity syntactically — every term a monomial in distinct variables, or the corner rule refuses to apply — and evaluates the 2^n corners in exact rationals; the enclosure is then the exact range, not an outer bound, and the two corners at its ends are the witnesses of a split refusal.

4.5 A sign, exactly

Geometric predicates — which side of an edge a point lies on — are signs of small determinants. Each is evaluated first in double precision and trusted only when its magnitude clears a forward-error bound proved for the inputs [7, 4]: with coordinates bounded by M and $u = 2^{-53}$, the computed 2×2 orientation determinant is within $48uM^2$ of the exact one. Below the bound, the sign is recomputed exactly in integers, each decimal literal scaled to a fixed sixty-four places; a literal that would need more is refused rather than rounded. A point on an edge is reported ON, an answer a floating-point test does not have. This is the static filter of exact geometric computation [34, 36], written against the literals a file contains rather than against their doubles. Over a contour benchmark it decided 6.37×10^8 predicates, 237,173 of them in integers [23].

4.6 A polynomial, nonnegative

A claim $p \geq \gamma$ is certified by an identity $p - \gamma = \sum_i c_i q_i^2$ with rational polynomials q_i and rational weights $c_i \geq 0$, checked by exact coefficient comparison; a negative weight or a nonzero residual refuses, and the residual is printed. Finding the decomposition is a semidefinite program [25, 26] that stays outside the trust path: the solver proposes, the identity decides. A separate refuter evaluates p at an exact witness point and refutes the claim if $p(w) < \gamma$.

4.7 A minimum of a cosine polynomial

The minimum over x of $\sum_{a \in A} \cos(ax)$ is the minimum over $y \in [-1, 1]$ of $P_A(y) = \sum_a T_a(y)$, a polynomial with integer Chebyshev coefficients. The certifier isolates the critical points with a Sturm chain of the square-free part of P'_A on a slightly enlarged interval, refines each by interval Newton in rationals with outward balls around exact Horner values, encloses P_A on each critical interval with the bound $a^2(a^2 - 1)/3$ of V. Markov on the second derivative, adds the endpoints

± 1 exactly, and returns the hull of the smallest enclosures. A stronger certificate identifies the minimum as an algebraic number — a resultant, a Sturm count of one inside the enclosure, irreducibility modulo a prime — and is CERTIFIED only when all of these hold.

4.8 Proposers

Every certifier above takes its candidate from a floating-point search — Nelder–Mead and Levenberg–Marquardt for a likelihood, Newton for a root, sampling for a minimum, a semidefinite solver for a decomposition, a language model for a construction — and none of these is ever in the trust path. A proposer may be wrong, slow or adversarial; it can cost a refusal, never a false verdict. The same rule governs forecasts in the system’s applications: a forecaster enters as a proposer of boxes, is graded against what is later observed, and stops being admitted when it misses its declared coverage.

5 The verification discipline

Proposition 1 makes the verdicts as sound as the code that computes the extensions. The discipline is how the system guards that code. Its rules are stated as engineering because each is a line of code that refuses — a battery that exits non-zero, a page builder that will not write its page, a numbers generator that will not write a paper’s macros — and this section says where each refusal sits.

Screens prune, never admit. Floating point decides only what is worth certifying. Every family of the generation loop declares a cheap screen that may only discard candidates and a certifier that is the only authority; the loop can skip on a screen and can record a hit only from the certifier. A screen is itself a claim that it drops nothing it should keep, so the campaign runner plants known answers that must pass every stage and certify before a run may start, refuses a run whose filters lose one, and requires a deliberately simple baseline beside every campaign so that a clever proposer is measured against something [15].

Red controls must fire. A check that has never gone red is decoration. A red control is a deliberately wrong variant of a test that must be rejected, and a battery whose red controls do not all fire fails, whatever its other checks say. Examples: a determinant whose double-precision value is $+3.6 \times 10^{-15}$ and whose exact sign is negative, which the filter must send to exact arithmetic; a “gate” that evaluates the floating-point model at the centre of the box, which is what a point pipeline does and which the corners must catch; a missing inner certificate, which must refuse a theorem; a bar on the wrong side of an endpoint, which must be refused by name. The test target runs 128 batteries, 82 in JavaScript and 46 in Python, and a wiring check refuses a tree in which that target and the control build’s list differ. In the six-lane audit of machine-written manuscripts, 21 mutation controls stand beside 92 named checks, and the campaign runner ships 19 ways to cheat its own reward signal, each caught by a named control [30]. The methods note records 16 defects the project shipped and then found: 10 caught by an impossible number, a calibration, a control, a byte pin or an outside read, and 6 by the adversarial read of the code made for this paper (Section 8), counted apart [15].

decision	the two implementations	recorded agreement
engineering gate	JavaScript intervals; Python decimal at fifty digits	inside the gate’s enclosures at 176 inputs, tightest margin 9.0×10^{-12} bar
abatement kernel	JavaScript exact rationals; Python fractions	8 scenario versions character for character, 2,400 interior points
wave-window decisions	JavaScript exact rationals; a clean-room Python verifier from the written specification	every published decision of a day re-decided, the build refusing on one disagreement
δ_3 certificates	the claimant’s Python verifier; a clean-room JavaScript verifier	5 of 5 certificates verified by both [3]
a Hankel-determinant wall	a lifted arbitrary-precision engine; a standard-library cross-check	51 rows re-run bit-identical, 6 of 6 cross-checks agreeing

Table 2: Decisions re-derived by a second implementation that shares no code with the first — written by the same author from the same specification, so an N-version check rather than an independent re-derivation. Each agreement is read from the record that holds it.

Pin the bytes. Every claim is decided against a held byte sequence named by path and SHA-256 and re-hashed at decision time: the claimant’s file, the dataset, the printed table transcribed digit for digit, and the certifying modules themselves, whose digests a ledger records and whose workers refuse other bytes. A page that reports a record refuses to build when a pinned input has changed, when the record was written by a different version of its verifier, or when its battery does not pass; an application build that regenerates a record and finds it different from the committed one refuses and leaves the committed record in place. Files lifted from the laboratory this system grew out of come through a manifest of their source paths and digests.

Independence from the claimant. The claimant’s program is never in the trust path: the decision is re-derived from the published statement and the published bytes, and where a claimant’s script was read to settle which identity a manuscript meant, the record says so and the script was not executed. An unqualified “no shared code” would not survive a check, because two of the system’s own instruments reuse its own code across a producer–checker line; the precise form adopted is that the system *shares no code with the claimant*, and every certificate re-runs with no engine present [27].

Same-author second implementations. Where a verdict is the product, a second program written from the specification, in another language and arithmetic and sharing no code, re-derives it (Table 2). These are N-version checks by the same author from the same specification on the same inputs: they catch a slip in one arithmetic and cannot catch a shared misreading, and they are counted apart from re-derivations by other parties (Section 6), of which the record holds 1. Not every certifier has one; Section 8 names those that do not.

Records re-run. A record is not believed because it was written; it is re-derived. Published records re-run in the reader’s browser by the bytes that wrote them and are compared with the record — by digest for a day of wave-window decisions and a cell of the extreme-value atlas, field

by field for an engineering receipt and an abatement registry. A rerun kit lists 31 commands that re-derive the register’s records or verify them with the standard library alone; its one recorded execution, on 2026-10-02, ran 23 of them in about 22 minutes on a laptop and compared 88 register rows before and after: 0 moved, 0 failed. And the system’s own results are offered to outside parties to re-certify without its code; 1 such rerun is recorded so far, the $\lambda(4)$ audit of Section 6.

Honest counting. Every row is derived from a record; a claim with no record gets no row. An aggregate audit counts once and names its most frequent defect. A correction the system adds beside a refuted claim is never counted among the audited corpus’s rows; a queued row is not a decided row; refusal kinds are never merged into one total; and every number in this paper and its companions is a macro written by a generator that refuses when a record no longer supports the sentence that quotes it [30].

6 The record

The system’s evidence is what it has decided, and every number below is a macro written from the certificate that decided it.

Machine-generated mathematics. The public register holds 132 published claims from 25 sources, decided between 2026-08-25 and 2026-10-05: 107 certified, 2 refuted, 9 partial, 5 mixed, 6 repaired, 2 needing data the claimant has not published, and 1 queued and not counted [30]. Most claims hold: 107 of the 131 decided rows name nothing wrong. The defects that appear are of the verifier’s kind rather than the mathematician’s. A constant posted to a problem thread as a bare decimal is a floating-point product, and parts from the certified enclosure at its 12th decimal place, by a witness that needs one integer inequality over the exact partial product of the primes up to 400,000; witnesses hold only within a verifier’s tolerance in 5 rows; one checker accepts what its own definition excludes. One row keeps as its history a clause present in a paper’s theorem and absent from its formal statement: OpenAI’s Navier–Stokes certificate read PARTIAL at the announced commit 8937a8f4, built here over 2,486 modules with only the standard axioms, and CERTIFIED at the next upstream commit f9e8bc5b, which proved the clause [19].

The laboratory’s own theorems. The method runs both ways. The value $\delta_3 = 117/2192$ [3] rests on certificates that two programs sharing no code re-decide, under a battery of 76 checks with 40 red controls. The value of $\lambda(4)$ [11] decides 2,231 finite sets across 9 families; on 2026-09-30 an outside audit with no shared code [12] re-derived the argument from the write-up, re-certified every finite case in interval arithmetic and swept the gcd-reduced sets up to 80 as a proof-independent control; its three documentation findings were folded in and no step broke. That audit is recorded as a row like any other.

Engineering numbers. A recommendation to raise water injection by 4.2% on a deep-water flowline, plausible to a reader, is REFUSED [24]: inputs in the declared envelope give wellhead pressures of 361.6 and 344.2 bar against a limit of 360 bar, each proved at a corner, and the rule is PROVED at a pump discharge of 204.0 bar or less. Of 7 injected faults of the kinds an industrial verifier must catch, 6 are refuted and 1 refused, each with its reason. A Monte Carlo study of

domain	what is decided	the check that shares no code
mathematics	132 published claims: constants, constructions, bounds, formal statements	the claimant’s checker is never run; one theorem of the laboratory re-certified by an outside group
engineering	an operating recommendation over a declared envelope; 7 injected faults	a fifty-digit decimal implementation, inside the enclosures at 176 inputs
metocean	46,602 likelihood fits; 176 printed contour counts	a library’s two answers decided as claims; the counts re-derivable in any exact arithmetic

Table 3: The record by domain. Every entry is a row of a public ledger that the build re-derives; the companion papers report each in full [30, 24, 23].

the same recommendation finds exceedances in 0.3% of 10,000 uniform draws with a worst draw of 360.7 bar against the proved worst corner of 361.6 bar — the sampling found the risk, and the decision states its extent without a prior over the envelope. A second implementation in fifty-digit decimal arithmetic, sharing no code, lies inside the gate’s enclosures at 176 inputs.

Metocean statistics. Over the open-sea cells of a public global hindcast, 40,778 of 46,602 maximum-likelihood fits are certified maxima, and 5,751 generalized-gamma fits are refused with the proof of Proposition 5; a scientific library’s default call is the maximum it reports on none of 60 fits. Of the 176 numbers a contour benchmark printed, 173 are the exact count, two are traced to files rewritten after the count, and 30 of its 150 polygons cross themselves [23].

7 Relation to existing practice

Validated numerics and computer-assisted proof. Interval arithmetic [16], interval methods for nonlinear systems [17], verification methods in floating point [33, 32], ball arithmetic [9] and the computer-assisted proofs they made possible [35, 6] are the certifiers’ ancestry, and the system uses them as such. What it adds is not a technique but a setting: the computations are aimed at other parties’ claims, the outcome is one of three published words with a witness, and a failure to decide is a result.

Formal proof. A proof assistant checks a proof against a stated theorem [13, 14]; what it cannot check is whether the statement is the one intended, and a translation that compiles may not be faithful [2]. The system complements formal proof in two ways: it decides the finite and numerical claims that formal libraries do not yet reach, and it audits formal claims at the level where they can fail — the statement read hypothesis by hypothesis, the build reproduced, the axioms asked of the kernel, the proof replayed by an independent kernel [19].

Exact geometric computation. Floating-point filters with exact fallbacks are standard in computational geometry [4, 34, 36]; the contribution here is only to apply them where a benchmark’s number is decided, and to keep ON as an answer.

Verification, validation and uncertainty quantification. Verification and validation qualify a model; uncertainty quantification gives a dispersion [20]. Neither decides whether a given

number holds for every input the engineer has declared, which is the question the epistemic reading of Definition 2 answers; the two are complementary, and the demonstrator sets them side by side [24].

Machine discovery. Systems that propose mathematics — conjectures from numerical coincidences [29], constructions from program search [31, 18] — each come with an evaluator written by the producer. The system sits downstream of all of them as an independent decider, and the register is the measure of what that independence finds.

8 What is not claimed, and the limits

Not claimed. Interval arithmetic, the Krawczyk operator, floating-point filters, sums of squares and Sturm sequences are not claimed; they are cited. The validity of a declared model is not claimed: a PROVED engineering number holds for the declared model over the declared envelope, not for the sea or the reservoir. Nothing is certified in the regulatory sense; the word is used in the mathematical sense of Definition 3 only. Nothing is a probability.

Locality and incompleteness. A certified maximum is the maximum in its box; a global statement over a family would need a branch-and-bound over the family’s whole parameter space, which the system does not yet run. The system refuses what it cannot decide, and the refusal rate is a property of each campaign, published with it.

Soundness rests on code. Proposition 1 is a theorem about interval extensions; whether a program computes one is a fact about the program. The defences — second implementations, red controls, pinned bytes, outside re-runs (Section 5) — reduce that risk and do not remove it, and they are not uniform: some certifiers have a second implementation that shares no code, and some, among them the point-in-polygon decider, the cosine-minimum certifier and the likelihood certifier, are re-run in the browser by the same bytes, which checks the record and not the code.

What a read of the code for this paper found, and what was closed. Two reads were made for this paper, the author’s on 2026-10-08 and an adversarial one on 2026-10-09; the second found defects under the system’s headline sentence, and they were closed the same day with the correction published beside the old numbers (the repository’s record `certs/erratum-2026-10-09.json` pins the numbers, the commit and the page bytes that carried them). *Closed:* the closed-form hunt of the generation loop compared a floating-point candidate — a square root, a constant times a rational, a power — against the certified enclosure and counted a miss as a refutation, exact for a rational candidate and not for the rest, so that 54.6 million of the loop’s “refutations” were float comparisons; every candidate is now an exact rational or an enclosure and a refutation is a disjointness, with the same vocabulary and a re-run of the whole corpus. The loop’s grammar had a word, REJECT, that covered both a proved failure and an enclosure straddling the bar, and any unknown word fell into it; one module now holds the three words and the one predicate of Definition 3, a straddle is REFUSED, an unknown word is REFUSED and counted (71 of the cosine family’s 1,579 decisions were straddles so counted). One family certified “exceeds the champion” by comparing lower end with lower end; the bar is now the champion’s enclosure. The interval kernel had no NaN or infinity semantics and a power routine that wrapped at exponents of 2^{31} ; both refuse. The linear branch of the radii-polynomial certifier returned success

without the interval check its other branch makes, and accepted a negative Lipschitz bound; the certificate constructor accepted empty falsifiers and undefined evidence; the aggregate test target finished successfully whatever its batteries printed and the control page was written with a red battery shown. All of these refuse now, and none had moved a verdict reported in the companion papers. *Open, and named*: the helper that encloses the platform’s cosine by a derived pad is called from six files including the radii-polynomial certifier of Section 4, whose records are re-run when it is retired; one quadrature routine that is rigorous on dyadic cells of $[0, 1]$ is called on another interval by one instrument; one battery labels some positive assertions as red controls; in the engineering demonstrator a PROVED consistency check means that the claim meets an outer enclosure, and the discharge thresholds are rounded outward to a tenth of a bar in floating point rather than in directed rounding; the transcendental tails are computed in round-to-nearest and are safe by a margin of twenty orders rather than by directed rounding; the same-author second implementations of Table 2 are not independent re-derivations; and the exact-class certificates alone are in a form a program other than the engine can check. Each is recorded in the repository’s debt register with the instrument it affects, and the system’s own rule applies to them — a check that has not been made is not a check.

9 Conclusions

cert-machine decides the numbers and claims that machines propose with three verdicts, each carrying the object that justifies it, sound under one hypothesis that the arithmetic is built to secure. Its certifiers reduce the questions that arrive — a zero, a maximum, a statement for every input, a sign, a nonnegative polynomial, a minimum — to finitely many exact or outward-rounded decisions; its discipline keeps them honest by code that refuses; and its record, 132 published claims, an engineering procedure and an extreme-value atlas, is public and re-runs. What it offers those who produce such claims is not a second opinion but a different object beside their number: a verdict with its witness, in which a refusal says what would decide it.

Reproduction

Every constant in this paper is interpolated from a certificate record in the **cert-machine** repository, so the document cannot quote a number the machine did not certify. The macros are written by `tools/paper-numbers/cert-machine-method.js` and by the generators of the companion papers (`tools/build-paper-numbers.js`, `tools/paper-numbers/register.js`, `ec-benchmark.js`, `delta3.js`) from the ledgers they name; a record that no longer supports a sentence makes the build refuse. Repository: <https://github.com/carlostoledo1891/cert-machine>, commit `be82e126bc6b`.

References

- [1] M. Abramowitz, I. A. Stegun, *Handbook of Mathematical Functions*, National Bureau of Standards, Applied Mathematics Series 55, 1964.
- [2] A. Bastounis, F. Circelli, A. C. Hansen, Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs, arXiv:2610.08144, 2026.

- [3] C. Toledo, The twelve-block colouring is optimal: $\delta_3 = 117/2192$ — a computer-assisted proof, its certificates re-decided by two programs that share no code, draft, 2026; archived at doi:10.5281/zenodo.23171167.
- [4] S. Fortune, C. J. Van Wyk, Static analysis yields efficient exact integer arithmetic for computational geometry, *ACM Trans. Graphics* 15 (1996) 223–248.
- [5] D. Goldberg, What every computer scientist should know about floating-point arithmetic, *ACM Computing Surveys* 23 (1991) 5–48.
- [6] T. Hales et al., A formal proof of the Kepler conjecture, *Forum Math. Pi* 5 (2017) e2.
- [7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.
- [8] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019.
- [9] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Computers* 66 (2017) 1281–1292.
- [10] R. Krawczyk, Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken, *Computing* 4 (1969) 187–201.
- [11] C. Toledo, The value of $\lambda(4)$: a machine-derived proof completing Mercer’s Section-5 program, <https://carlostoledo.co/reports/lambda4.html>, doi:10.5281/zenodo.22225861 (2026).
- [12] rainrzk, erdos510-lambda4-audit, <https://github.com/rainrzk/erdos510-lambda4-audit>, posted on <https://github.com/teorth/erdosproblems/issues/392> (2026-09-30).
- [13] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in: *CADE-28*, LNCS 12699, Springer, 2021, 625–635.
- [14] The mathlib Community, The Lean mathematical library, in: *CPP 2020*, ACM, 2020, 367–381.
- [15] How the bugs were found: the methods note (formerly ”None by reading code”; on 2026-10-09 an adversarial read of the code found six, counted apart), <https://carlostoledo.co/reports/methods-note.html>.
- [16] R. E. Moore, R. B. Kearfott, M. J. Cloud, *Introduction to Interval Analysis*, SIAM, 2009.
- [17] A. Neumaier, *Interval Methods for Systems of Equations*, Cambridge University Press, 1990.
- [18] A. Novikov et al., AlphaEvolve: a coding agent for scientific and algorithmic discovery, arXiv:2506.13131, 2025.
- [19] C. Toledo, An audit of the 2026 Navier–Stokes finite-time blowup claim: the formal statement read against Clay’s wording and Mathlib’s definitions, the Lean proof built and its axioms asked of the kernel, draft, 2026; report at <https://carlostoledo.co/reports/navier-stokes.html>.
- [20] W. L. Oberkampff, C. J. Roy, *Verification and Validation in Scientific Computing*, Cambridge University Press, 2010.
- [21] T. Ogita, S. M. Rump, S. Oishi, Accurate sum and dot product, *SIAM J. Sci. Comput.* 26 (2005) 1955–1988.
- [22] OpenAI, NavierStokesAndEuler, <https://github.com/openai/NavierStokesAndEuler>, commits 8937a8f4 and f9e8bc5b, 2026.

- [23] C. Toledo, Certified extreme-value analysis of significant wave height: maximum-likelihood fits and return levels over a public global hindcast, and environmental contours decided exactly, draft v0.2, 2026; <https://carlostoledo.co/paper/p1-return-levels.pdf>.
- [24] C. Toledo, A decision procedure for machine-proposed engineering numbers: three-valued verdicts over a declared uncertainty envelope, with witnesses and the threshold that decides, draft, 2026; <https://carlostoledo.co/paper/p2-decision-procedure.pdf>.
- [25] P. A. Parrilo, Semidefinite programming relaxations for semialgebraic problems, *Math. Programming* 96 (2003) 293–320.
- [26] H. Peyrl, P. A. Parrilo, Computing sum of squares decompositions with rational coefficients, *Theoret. Comput. Sci.* 409 (2008) 269–281.
- [27] Positioning: the decisions, [notes/positioning-decisions-2026-09-03.md](https://github.com/carlostoledo1891/cert-machine), <https://github.com/carlostoledo1891/cert-machine>.
- [28] R. L. Prentice, A log gamma model and its maximum likelihood estimation, *Biometrika* 61 (1974) 539–544.
- [29] G. Raayoni et al., Generating conjectures on fundamental constants with the Ramanujan Machine, *Nature* 590 (2021) 67–73.
- [30] C. Toledo, Independent exact certification of machine-generated mathematics: the first register, draft, 2026; <https://carlostoledo.co/paper/register.pdf>.
- [31] B. Romera-Paredes et al., Mathematical discoveries from program search with large language models, *Nature* 625 (2024) 468–475.
- [32] S. M. Rump, INTLAB — INTerval LABoratory, in: T. Csendes (ed.), *Developments in Reliable Computing*, Kluwer, 1999, 77–104.
- [33] S. M. Rump, Verification methods: rigorous results using floating-point arithmetic, *Acta Numerica* 19 (2010) 287–449.
- [34] J. R. Shewchuk, Adaptive precision floating-point arithmetic and fast robust geometric predicates, *Discrete Comput. Geom.* 18 (1997) 305–363.
- [35] W. Tucker, A rigorous ODE solver and Smale’s 14th problem, *Found. Comput. Math.* 2 (2002) 53–117.
- [36] C. K. Yap, Towards exact geometric computation, *Comput. Geom.* 7 (1997) 3–23.